

Placement portal: AWS + Vercel — problem and fix

Hand this to whoever can change the **AWS instance**. Vercel env changes are included so both sides stay in sync.

Priority: Admin assessments must work (create, student takes test, live monitor, pause/extend, webcam signaling).

1. Current state (today)

The live frontend is on Vercel. The backend is on AWS. There is **no custom domain** for the API.

How the frontend is configured to talk to AWS right now:

```
VITE_API_BASE_URL=http://3.109.81.183:3000/api
VITE_SOCKET_URL=http://3.109.81.183:3000
VITE_FRONTEND_URL=https://setuioi.vercel.app
```

Piece	Where	Actual URL
UI	Vercel	https://setuioi.vercel.app
API	AWS EC2, Node on port 3000	http://3.109.81.183:3000/api
Sockets	same Node process	http://3.109.81.183:3000 (path /socket.io)
Health	same	http://3.109.81.183:3000/health → { "status": "ok" }

What is already true on AWS:

- Node is running and reachable on **HTTP port 3000**.
- [GET /health](#) works.
- CORS already allows origin <https://setuioi.vercel.app> (preflight from that origin returns [Access-Control-Allow-Origin](#)).

What is **not** true:

- The API is **not HTTPS**.
 - There is no hostname/certificate in front of Node.
-

2. The whole problem

What users see

On <https://setuioi.vercel.app>:

Failed to connect to server. Cannot reach the server.

Network tab: requests to `http://3.109.81.183:3000/api/...` fail.

Why (mixed content)

1. The website is **HTTPS** (`https://setuioi.vercel.app`).
2. The API URL in env is **HTTP** (`http://3.109.81.183:3000`).
3. Browsers **block** an HTTPS page from calling an HTTP API. That is mixed content.
4. The JavaScript never gets a response, so the app says the server is unreachable.

This is **not**:

- Node being down (health on the IP still works in a new tab).
- A missing `/api` index route (GET `http://3.109.81.183:3000/api` returning "Route not found" is normal).
- A CORS bug (CORS already allows the Vercel origin when you call HTTP from curl). CORS never gets a chance in the browser because mixed content blocks first.

Why `https://3.109.81.183` will not fix it

Someone might think: "just use https on the IP."

That will **not** work:

- You cannot get a normal trusted Let's Encrypt certificate for a **raw IP**.
- A self-signed cert shows a browser warning; `fetch` and Socket.IO still fail.
- Admin assessments also need **WebSockets**. They need a proper HTTPS hostname, not `https://3.x.x.x`.

Why a Vercel proxy is not enough for assessments

Proxying `/api` through Vercel can make **simple REST** (login, job lists) work, because the browser only talks HTTPS to Vercel.

Admin assessments also need:

- Socket.IO (live monitor, pause/extend while a student is in the exam)
- WebRTC signaling for live webcam

Vercel rewrites do **not** carry that reliably. For assessments, the browser must talk **directly** to HTTPS on AWS.

3. The whole solution

Keep Node on port 3000 as it is today.

Install **Caddy** on the same instance:

- Caddy listens on **80** and **443**.
- Caddy forwards to `127.0.0.1:3000`.
- Use a **free hostname** that already points at this IP (no domain purchase):

```
https://3-109-81-183.sslip.io
```

`3-109-81-183.sslip.io` resolves to `3.109.81.183`. Caddy obtains a Let's Encrypt certificate automatically.

After AWS is done, public URLs become:

```
VITE_API_BASE_URL=https://3-109-81-183.sslip.io/api
VITE_SOCKET_URL=https://3-109-81-183.sslip.io
VITE_FRONTEND_URL=https://setuioi.vercel.app
```

Before (broken)	After (required)
<code>http://3.109.81.183:3000/api</code>	<code>https://3-109-81-183.sslip.io/api</code>
<code>http://3.109.81.183:3000</code>	<code>https://3-109-81-183.sslip.io</code>
<code>http://3.109.81.183:3000/health</code>	<code>https://3-109-81-183.sslip.io/health</code>

Then both REST and Socket.IO go over HTTPS to the same host. Assessments can work.

4. AWS instance steps (do these in order)

Step 1 — Security group

AWS console → this instance's security group → inbound:

Port	Protocol	Source	Required
80	TCP	<code>0.0.0.0/0</code>	Yes (certificate)
443	TCP	<code>0.0.0.0/0</code>	Yes (HTTPS)
3000	TCP	already open	Optional after Caddy; can stay for debug

If the box uses `ufw`:

```
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw status
```

Step 2 — Confirm Node (do not change the app yet)

```
curl -sS http://127.0.0.1:3000/health
curl -sS http://3.109.81.183:3000/health
```

Both should return JSON with `"status": "ok"`. If localhost fails, start/fix Node first. Caddy only reverse-proxies to it.

Step 3 — Install Caddy (Ubuntu)

```
sudo apt update
sudo apt install -y caddy
```

Step 4 — Caddyfile

Write **exactly** this to `/etc/caddy/Caddyfile`:

```
3-109-81-183.sslip.io {
    reverse_proxy 127.0.0.1:3000
}
```

If Node is only reachable via Docker on another host port, change `127.0.0.1:3000` to wherever port 3000 is published on the host.

```
sudo caddy validate --config /etc/caddy/Caddyfile
sudo systemctl enable --now caddy
sudo systemctl reload caddy
sudo systemctl status caddy --no-pager
```

Do **not** stop Node. Do **not** move Node off port 3000.

Step 5 — Backend env on the instance

In the **AWS** backend `.env` (not the laptop local file):

```
FRONTEND_URL=https://setuioi.vercel.app
CORS_ORIGIN=https://setuioi.vercel.app
```

Rules:

- No trailing slash
- No quotes
- Must be `https://setuioi.vercel.app`, not the sslip.io URL

Restart Node after saving:

```
pm2 restart all
# or: sudo systemctl restart <node-service>
# or: docker compose restart <api-service>
```

Step 6 — Prove HTTPS from the instance

```
curl -sS -i https://3-109-81-183.sslip.io/health
```

Expect HTTP 200 and **no** SSL error.

CORS check:

```
curl -sS -i -X OPTIONS 'https://3-109-81-183.sslip.io/api/cms/public/landing?page=landing' \
-H 'Origin: https://setuioi.vercel.app' \
-H 'Access-Control-Request-Method: GET'
```

Expect **Access-Control-Allow-Origin: https://setuioi.vercel.app**.

From a laptop browser, open:

<https://3-109-81-183.sslip.io/health>

Padlock, no certificate warning.

If the cert fails, port 80 is usually closed. Logs:

```
sudo journalctl -u caddy -n 80 --no-pager
```

5. Vercel steps (after AWS HTTPS works)

AWS cannot finish the job alone. **VITE_*** is compiled into the frontend at **build** time.

In Vercel → Project → Settings → Environment Variables, **replace** the current AWS HTTP values:

```
VITE_API_BASE_URL=https://3-109-81-183.sslip.io/api
VITE_SOCKET_URL=https://3-109-81-183.sslip.io
VITE_FRONTEND_URL=https://setuioi.vercel.app
```

Remove any of:

```
VITE_API_BASE_URL=http://3.109.81.183:3000/api
VITE_SOCKET_URL=http://3.109.81.183:3000
```

and any Render URL (*.onrender.com).

Then **Redeploy**. Saving env without a new build leaves the old HTTP IP in the JS bundle.

6. What “done” looks like

On <https://setuioi.vercel.app> DevTools → Network:

- Calls go to <https://3-109-81-183.sslip.io/api/...> (200)
- Socket.IO goes to <https://3-109-81-183.sslip.io/socket.io/...>
- **No** <http://3.109.81.183:3000>
- **No** mixed-content errors in the console

Admin assessment checks:

1. Admin creates / assigns an assessment.
2. Student starts and submits.
3. Admin live monitor updates.
4. Admin pause / extend during the exam.
5. Webcam / live proctor signaling.

If lists load but the live monitor is empty, [VITE_SOCKET_URL](#) is wrong or Caddy is not proxying [/socket.io](#) (the Caddyfile above proxies everything).

7. Troubleshooting

Symptom	Cause	Fix
Site: cannot reach server; IP /health works in a new tab	Mixed content	Finish Caddy + change Vercel to sslip.io HTTPS + Redeploy
https://3.109.81.183 cert warning	HTTPS on raw IP	Use https://3-109-81-183.sslip.io only
sslip.io cert warning	Port 80 closed or Caddy down	Open 80/443; check <code>journalctl -u caddy</code>
<code>curl http://127.0.0.1:3000/health</code> fails	Node down	Start Node before Caddy can help
CORS error after HTTPS works	<code>CORS_ORIGIN</code> mismatch	Set exact https://setuioi.vercel.app , restart Node

Symptom	Cause	Fix
Site still calls <code>http://3.109.81.183</code>	Old Vercel build	Update env, Redeploy
Site still calls Render	Old Vercel env	Delete onrender.com vars, Redeploy
API works, live monitor does not	Sockets not on HTTPS host	<code>VITE_SOCKET_URL=https://3-109-81-183.sslip.io</code> (no <code>/api</code>)

8. Do not do this

- Do not switch the frontend to `https://3.109.81.183:3000`.
- Do not put a self-signed cert on Node port 3000 and call it done.
- Do not stop Node or change it off port 3000; Caddy sits in front.
- Do not leave `VITE_API_BASE_URL=http://3.109.81.183:3000/api` after Caddy is up.
- Do not rely on a Vercel `/api` proxy for admin assessments.

9. Checklist

AWS

- ☐ Inbound 80 and 443 open
- ☐ `curl http://127.0.0.1:3000/health` OK (current Node, unchanged port)
- ☐ Caddy installed; Caddyfile = `3-109-81-183.sslip.io → 127.0.0.1:3000`
- ☐ `https://3-109-81-183.sslip.io/health` works with a padlock
- ☐ Instance `FRONTEND_URL` and `CORS_ORIGIN` = `https://setuioi.vercel.app`
- ☐ Node restarted after `.env` change

Vercel (after AWS HTTPS is proven)

- ☐ `VITE_API_BASE_URL=https://3-109-81-183.sslip.io/api`
- ☐ `VITE_SOCKET_URL=https://3-109-81-183.sslip.io`
- ☐ `VITE_FRONTEND_URL=https://setuioi.vercel.app`
- ☐ Redeployed
- ☐ Admin assessment live monitor tested